# Selecting Reliable LLM Reasoning with a Compact V3 Ensemble

DASE7506 Mini Project 2 | 31 October 2026

ZHANG YIYI 3036779970

**Abstract**

This project selects a correct numeric answer from four supplied language-model solutions to a mathematics problem. I study whether candidate representations, observable reasoning features and comparisons among candidates can improve selection under a 16 MiB asset limit and a 30-second CPU evaluation budget. The final V3 method combines three small multilayer perceptrons with shallow LambdaRank trees, then averages scores for candidates sharing the same parsed answer. It reaches 754/798 correct on validation (94.49%) and 382/467 on the original test bank (81.80%). The frozen assets occupy 1.766 MiB; an external local prediction-plus-scoring measurement is 1.231 seconds. Validation ablations show that adding the ranker corrects five additional questions and answer pooling contributes nine correct choices relative to otherwise identical inference without pooling. The remaining failures often involve semantic conditions that are not resolved by valid arithmetic alone.

## 1 Task and evaluation

The input is a question, four candidate solution texts and a fixed 1024-dimensional representation for each candidate. The output is one candidate index from 0 to 3. More than one candidate may be correct, so all candidates matching the reference numeric answer are positive training examples. The official parser treats equivalent fractions and decimals consistently. The scoring target is final-answer correctness, not the validity of every reasoning step [1].

| Split | Questions | Candidate solutions |
|---|---|---|
| Training | 4,755 | 19,020 |
| Validation | 798 | 3,192 |
| Test | 467 | 1,868 |

The central hypothesis is that complementary scoring models and comparisons between candidates provide useful evidence beyond evaluating each candidate independently. A second hypothesis is that accuracy on questions with genuinely different candidate answers is more informative than aggregate validation accuracy. I therefore report both the full validation bank and its disagreement subset throughout the analysis.

The submitted recipe uses only the supplied training inputs and references to fit its parameters. The final training script loads training and validation splits; its validation labels are used for reporting and export checks. The frozen predictor reads no reference files and performs no generation, API calls or answer lookup.

# 2 Evidence represented by V3

## 2.1 Fixed representations and relative comparisons

Each supplied embedding is L2-normalized. For every candidate, I concatenate the normalized embedding with its difference from the mean of the four embeddings for that question. The resulting 2048 values encode both the individual solution and its position relative to the alternatives. The subtraction is invariant to a reordering of the four candidates and does not use correctness labels.

## 2.2 Observable text and numeric features

V3 adds 47 deterministic text features per candidate and their 47 within-question deviations, producing 2142 input features in total. These features expose evidence that a pooled representation may not preserve explicitly. They do not constitute a symbolic proof system.

| Feature family | Examples and purpose |
|---|---|
| Presentation | Log counts of characters, words, lines, numbers, equalities and formatting markers. |
| Arithmetic evidence | Counts of accepted and rejected explicit numeric equalities, and the rejection fraction. |
| Final answer | Parse validity, sign, zero and integer indicators, and a bounded signed magnitude. |
| Agreement and question relation | Answer frequency, distinct answer count, numeric overlap and lexical overlap with the question. |
| Reasoning vocabulary | Counts of 20 phrases such as however, approximately, remaining, total and check. |

The arithmetic checker first normalizes a small set of multiplication, division and fraction notations. It accepts only bounded numeric abstract syntax trees containing addition, subtraction, multiplication and division. It compares the computed value with the stated result using the implemented tolerance. Unsupported syntax is skipped; a skipped check is not evidence that a statement is correct.

## 2.3 Why these features can help and mislead

Length and formatting may correlate with correctness in this candidate bank, but they can also reflect generation style. Numeric overlap may identify relevance or merely reward copying numbers from the question. Arithmetic checks can detect local contradictions while missing a wrong interpretation of the task. Combining these features with embeddings is therefore a prediction strategy whose usefulness must be tested, rather than a claim to understand mathematical reasoning.

Means and standard deviations are fitted on training features only. Each dimension is standardized using a standard-deviation floor of 0.0001. The same saved statistics are used at validation and inference. Candidate IDs are used only by the surrounding tools for alignment; they are not model features.

# 3 Training and final decision rule

## 3.1 Small neural scorers

Three scikit-learn MLP classifiers use seeds 2026, 2027 and 2028. Each has a single 64-unit ReLU hidden layer and a binary output. A candidate is labelled positive when its parsed answer matches the reference. Training consists of six partial_fit passes with Adam. Questions with at least two distinct valid candidate answers receive weight 1; other questions receive weight 0.25, repeated across their four candidates. This emphasizes informative comparisons without discarding the rest of the training bank [2].

| Neural parameter | Value |
|---|---|
| Hidden units | 64 |
| Passes and batch size | 6 passes; 128 candidates |
| Initial learning rate | 0.0005 |
| L2 regularization alpha | 10 |
| Seeds | 2026, 2027, 2028 |
| Disagreement and other weights | 1 and 0.25 |

## 3.2 A complementary group ranker

A LightGBM model uses the LambdaRank objective, with each four-candidate question forming a ranking group [3]. It fits only the 2114 disagreement training questions. Binary relevance labels allow several correct candidates within a group. The fixed configuration uses 400 rounds, seven leaves, learning rate 0.03, minimum 80 observations per leaf, L2 penalty 10, maximum 63 bins and feature fraction 0.8. Its seed is 2026 and it uses four CPU threads.

## 3.3 Fusion and answer pooling

At inference, each of the three MLP raw logits and the tree raw score is centered within its question, then assigned weight 0.25. The three MLPs therefore contribute 75% and the ranker 25%. These are raw-score fusion weights, not calibrated probability weights. Centering removes offsets shared by all four candidates but does not calibrate the scales across models.

For each distinct parsed final answer, V3 replaces the scores of all candidates giving that answer with their mean. Invalid parsed answers receive negative infinity. The highest resulting candidate score wins; ties follow candidate order. Mean pooling tests agreement without automatically giving a larger answer group a sum-based advantage. There is no additional vote bonus.

All neural weights and tree nodes are exported to NumPy arrays. Inference therefore requires NumPy and the unchanged official parser, not scikit-learn or LightGBM. Maximum recorded neural probability export error is 1.19e-7; tree raw-score error is zero on validation.

# 4 Comparison experiments

Table 1 compares the ten reported validation entries using the same 798 questions and the official scorer. The selected initial method duplicates the combined ridge model and is retained for traceability; it is not an independent algorithm. All figures below are measured results, not expected performance.

**Table 1 Validation comparison in descending accuracy order**

| Method | Correct | Accuracy |
|---|---|---|
| V3 neural and ranking ensemble | 754 / 798 | 94.4862% |
| Hashed text model | 740 / 798 | 92.7318% |
| Ridge and MLP fusion with answer pooling | 737 / 798 | 92.3559% |
| Hard rank on disagreement groups | 736 / 798 | 92.2306% |
| Three seed MLP ensemble | 733 / 798 | 91.8546% |
| CatBoost with arithmetic features | 722 / 798 | 90.4762% |
| Combined feature ridge | 720 / 798 | 90.2256% |
| Initial selected method same as combined ridge | 720 / 798 | 90.2256% |
| Embedding ridge | 716 / 798 | 89.7243% |
| Majority answer baseline | 688 / 798 | 86.2155% |

## 4.1 What the comparisons establish

The majority baseline is strong: repeated final answers already contain useful evidence. Ridge on fixed embeddings adds 28 correct choices. Adding the initial small text feature group yields four further choices, although its best regularization differs, so this is a feature-family comparison with validation-selected regularization rather than an exact fixed-parameter ablation.

The MLP ensemble improves over combined ridge by 13 questions. A hashed word and adjacent-token model reaches 740 correct, whereas the more elaborate candidate-interaction hard-rank model reaches 736. Greater model complexity alone is therefore insufficient. CatBoost is weaker still, suggesting that local arithmetic and tree partitioning do not automatically replace the information captured by the neural representations.

V3 adds 14 correct choices over the strongest earlier validation entry, a gain of 1.7544 percentage points. Relative to majority voting, it improves by 8.2707 points and reduces validation errors from 110 to 44. This 60% reduction is descriptive for this bank; it is not evidence of an equivalent reduction on new questions.

The baseline entries differ in architecture, objectives, feature sets and search budgets. Their comparison establishes overall recipe performance. The controlled inference ablations on the following page isolate specific components of the final frozen recipe more directly.

# 5 Ablations and uncertainty

I evaluated inference ablations on the same frozen V3 assets and original validation bank. No weights were retrained. In each single-family variant, the retained models were renormalized to total weight 1. Pooling remained enabled except in the explicitly identified variant. These are component diagnostics rather than additional submitted models.

**Table 2 Controlled validation ablations of the frozen V3 checkpoint**

| Variant | All correct | All accuracy | Disagreement accuracy |
|---|---|---|---|
| Full V3 | 754 / 798 | 94.49% | 311 / 355 = 87.61% |
| MLPs only | 749 / 798 | 93.86% | 306 / 355 = 86.20% |
| Trees only | 742 / 798 | 92.98% | 299 / 355 = 84.23% |
| V3 without answer pooling | 745 / 798 | 93.36% | 302 / 355 = 85.07% |

Adding the ranker to the MLP-only ensemble fixes five validation choices and breaks none in this comparison. Conversely, trees alone are weaker than the full ensemble. This supports complementary decision information rather than replacing the neural scorers with the ranker.

Disabling pooling loses nine correct questions, a 1.1278-point drop overall and a 2.5352-point drop on disagreement questions. Thus, comparing candidates at the final-answer level contributes measurable value. It remains possible for several wrong candidates to reinforce the same interpretation; agreement is evidence, not a correctness guarantee.

## 5.1 Why disagreement accuracy matters

Only 355 of 798 validation questions have at least two distinct valid candidate answers. Full V3 gets 311 of these correct and all 443 other questions correct. The latter group therefore contributes 55.51% of the bank and raises aggregate accuracy. By contrast, the supplied test construction requires answer disagreement. Reporting 94.49% alone would obscure this difference. The remaining validation errors are concentrated entirely in the disagreement subset.

## 5.2 Score margins as an uncertainty signal

For disagreement questions, I measure the gap between the largest and second-largest pooled scores for distinct answers. Using distinct answers avoids zero gaps caused merely by two candidates sharing one pooled answer. Among the 88 lowest-margin questions, 66 are correct (75.00%); among the 88 highest-margin questions, 87 are correct (98.86%).

The gap is associated with accuracy on this validation sample, but it is not a calibrated probability. The quartiles are descriptive and were not used to tune an abstention rule. The required output always chooses one candidate. A future selective system would need separately validated thresholds and coverage reporting.

# 6 Failure analysis and generalization

V3 makes 44 validation errors and 85 test errors. The following cases come from validation, where all four candidate texts and references were inspected. They illustrate failure mechanisms; they are not an exhaustive annotation of the error bank.

### 6.1 Correct arithmetic with a wrong category

In the eggs question, 24 eggs are purchased, three break and six crack. The selected candidate correctly computes 15 perfect eggs, then incorrectly calls three plus six the cracked count and returns six. The reference difference is nine. Several equalities in the selected trace are numerically true, so an arithmetic checker cannot detect that the wrong category has been substituted. This demonstrates a limit of local equality features.

### 6.2 Counting pairs as people

The dancing question has 46 male and 48 female students. The selected candidate correctly obtains 94 students and 46 pairs, but subtracts 46 from 94 and returns 48 unpaired students. Each pair contains two students, leaving only two unpaired. The erroneous final subtraction is itself arithmetically valid. Recognizing units and relationships is more important here than checking calculation syntax.

### 6.3 Mentioning a condition without applying it

In the painting question, the selected trace acknowledges that a second coat is required but computes only four gallons for one coat, then divides the 180-dollar cost between two people and returns 90. Applying the second coat gives 180 per person. Surface mention of a condition does not establish that it affected the calculation. A useful next direction would connect quantities to explicit state changes or units, rather than add more vocabulary counts.

### 6.4 Interpreting the test result

The final official test score is 382/467, or 81.7987%. The 12.6875-point gap from overall validation accuracy partly reflects the different answer-disagreement composition. Comparing with disagreement validation accuracy gives a smaller 5.8069-point gap, but composition alone does not prove the cause of every difference. Sampling variation, problem difficulty and validation adaptation may also contribute.

This is an exploratory comparison on a public course bank, not a preregistered evaluation. The reported test value is a descriptive bank score rather than an unbiased estimate of arbitrary future performance. A separate unseen audit bank would provide stronger evidence of generalization. The scorer also measures final-answer correctness only: it cannot establish that every selected reasoning step is sound.

# 7 Computational cost and resource use

The final V3 recipe trains on CPU using scikit-learn and LightGBM. The RTX 3060 Laptop GPU was used by earlier PyTorch and CatBoost baselines, but it is not needed for V3 training or inference. Table 3 summarizes saved run records. These are local recipe costs with different search scopes, not a controlled hardware speed benchmark.

**Table 3 Recorded local training or selection costs**

| Recipe | Time in seconds | Saved asset size |
|---|---|---|
| Ridge search | 2.34 | Small linear parameter bundles |
| MLP search | 27.17 | 1.557 MiB |
| Fusion selection only | 0.76 | 1.582 MiB |
| Hard rank search | 22.82 | 1.200 MiB |
| Hashed text search | 153.06 | 1.817 MiB |
| CatBoost search | 130.85 | 0.259 MiB |
| Final V3 training and export | 18.31 | 1.766 MiB |

Fusion selection time excludes training its input models. Baseline search records include their own candidate configurations, whereas V3 records one fixed recipe. Therefore these timings should not be interpreted as equal-budget comparisons. The compact V3 result shows that increasing computation is not the only route to improvement.

## 7.1 Complete inference measurement

A separate external Stopwatch measured one fresh prediction process followed by one fresh official scoring process on all 467 test questions. Prediction took 0.970086 seconds, scoring took 0.260568 seconds, and their combined wall time was 1.230654 seconds. This includes process startup, imports, data and checkpoint loading, feature construction, all four ensemble members and scoring. Internal runner timings alone were not used for the budget comparison.

## 7.2 Asset accounting

The checkpoint contains config.json, three MLP arrays and rank_trees.npz. Their total file size is 1,851,655 bytes (1.766 MiB), including saved normalizers and configuration. Uncompressed arrays occupy 1,845,212 bytes. Both values are well below 16 MiB. Ordinary source and supplied data are reported separately. No pretrained weights, cached choices or question lookup index are required.

# 8 Reproducing the final submission

Run commands from the official code directory with unchanged common.py, predict.py, score_predictions.py and original data. The submitted predictor is submission_v3.py and its checkpoint directory is runs/v3_model. The training log records Python 3.12.4, NumPy 1.26.4, SciPy 1.17.1, scikit-learn 1.7.2 and LightGBM 4.6.0; the package environment records threadpoolctl 3.7.0. These differ from some suggested versions in source comments, so the actual recorded environment is the reproduction target.

## 8.1 Installation and training

```
python -m pip install numpy==1.26.4 scipy==1.17.1
python -m pip install scikit-learn==1.7.2 threadpoolctl==3.7.0
python -m pip install lightgbm==4.6.0
$env:OMP_NUM_THREADS='4'
$env:OPENBLAS_NUM_THREADS='4'
$env:MKL_NUM_THREADS='4'
$env:NUMEXPR_NUM_THREADS='4'
python check_data.py
python train_v3.py --output runs/v3_model
```

The training output directory must not already exist. To preserve the submitted checkpoint, retrain into a new directory and use that path consistently below. Training is unnecessary when reproducing the score from the supplied checkpoint.

## 8.2 Official prediction and scoring

```
python predict.py --method submission_v3.py --checkpoint runs/v3_model
  --split validation --output runs/v3-validation-predictions.json
python score_predictions.py --split validation
  --predictions runs/v3-validation-predictions.json
  --output runs/v3-validation-score.json
python predict.py --method submission_v3.py --checkpoint runs/v3_model
  --split test --output runs/v3-test-predictions.json
python score_predictions.py --split test
  --predictions runs/v3-test-predictions.json --output runs/v3-test-score.json
```

Each wrapped command above is one shell command; join its lines before execution. Expected frozen-checkpoint results are 754/798 on validation and 382/467 on the original test bank. Inference itself requires only NumPy and the official parser. The training dependencies are needed only for retraining.

## 8.3 Reproducing the inference ablations

Copy the checkpoint to separate diagnostic directories and edit only config.json. For MLP-only inference, retain the three MLP entries with weight one third each. For trees-only inference, retain the tree entry with weight 1. For no pooling, retain all original members and set pool to false. Keep vote_weight at zero. Run the same validation prediction and scoring commands with the diagnostic checkpoint path. Do not retrain or use the test bank for these component comparisons.

# 9 Conclusions and reproducibility record

The strongest recorded validation recipe is a compact combination of three regularized neural scorers, a shallow within-question ranker and mean pooling of equal parsed answers. It achieves 94.49% overall validation accuracy and 87.61% on disagreement validation questions. Its original-bank test result is 81.80%. The compact asset footprint and CPU-only implementation leave substantial local resource headroom, although reference-machine eligibility remains subject to course evaluation.

The controlled diagnostics support two concrete findings: the tree contribution fixes five validation choices relative to the retained MLPs alone, and answer pooling contributes nine choices. They do not isolate the causal benefit of each handcrafted feature, establish probability calibration or prove mathematical reasoning competence. Several failed candidates perform valid arithmetic on the wrong categories, units or conditions.

A useful extension would represent quantities and conditions explicitly, then test that representation under training-only internal folds and a separate audit bank. More seeds, larger hidden layers or more trees should be justified by reproducible gains and measured cost rather than assumed to improve reliability.

## 9.1 Frozen artifacts

The final checkpoint consists of config.json, mlp_2026.npz, mlp_2027.npz, mlp_2028.npz and rank_trees.npz. The training information JSON records seeds, parameters, versions, asset hashes, parity errors and training time. The official validation/test score JSONs and the external timing record should accompany the code repository. Public code and checkpoint links must identify the same frozen version.

**Table 4 SHA256 fingerprints for key frozen assets**

| Asset | SHA256 |
|---|---|
| config.json | 9c16f007fab11539b9a1041db8917015<br>ab48d5c180202784198c97c2f6afb800 |
| rank_trees.npz | 2b2de3f6311e790cafcb052bca1b9cd0<br>a9eb90466af76edb41c10158996132de |

## 9.2 AI assistance and attribution

OpenAI ChatGPT and Codex assisted with method design, implementation, debugging, experiment analysis and report preparation. The official data, parser and scoring tools are attributed to the course repository. Questions derive from GSM8K and supplied candidate traces and representations were prepared with Qwen3-0.6B as described in the data manifest. The supplied license and notice files should be retained. No additional pretrained weights or external training datasets are used by the submitted V3 recipe.

## References

[1] DASE7506 MP2. Project Guide and Code README. https://github.com/xudongwu-0/DASE7506-MP2

[2] scikit-learn 1.7. MLPClassifier documentation. https://scikit-learn.org/1.7/modules/generated/sklearn.neural_network.MLPClassifier.html

[3] LightGBM 4.6.0. Parameters documentation, ranking objectives and learning controls. https://lightgbm.readthedocs.io/en/v4.6.0/Parameters.html